Performance Implications of Transparent Huge Pages on Nginx Static Web Workloads under Varying Request Concurrency

Saurabh Pandey, Poojan Pandya
International Institute of Information Technology Bangalore
Bangalore, India
Email: saurabh.pandey@iiitb.ac.in, poojan.pandya@iiitb.ac.in

***Abstract*—Transparent Huge Pages (THP) are a Linux kernel memory optimization mechanism that automatically promotes standard 4 KB pages into larger 2 MB pages in order to reduce Translation Lookaside Buffer pressure and improve memory locality. While THP has shown promising benefits for memory-intensive applications, its impact on lightweight web server workloads remains comparatively underexplored. In this paper, we conduct an empirical study of THP on an Nginx-based static web server deployed on a Linux virtual machine. Two workload categories were considered: small static files (4 KB) and large static files (5 MB), each evaluated under client concurrency levels of 50, 200, and 400 using Apache Benchmark. For each experimental setting, repeated benchmark executions were performed under both standard 4 KB paging and THP-enabled memory modes while collecting application-level and system-level metrics through Apache Benchmark and Linux `perf`. We analyze throughput, request latency, page faults, and context switching behavior. Experimental observations indicate that THP consistently improves throughput by approximately 3–5% and lowers latency for small-file workloads, while reducing total page faults significantly under higher concurrency. For large-file workloads, THP still provides modest throughput gains but exhibits more variable page fault behavior, suggesting that huge page promotion and kernel memory compaction overhead can influence memory-intensive transfers. The study demonstrates that THP can offer measurable advantages for concurrent web serving, although the degree of benefit remains workload dependent.**

***Index Terms*—Transparent Huge Pages, Nginx, Linux Memory Management, Apache Benchmark, Page Faults, Web Server Performance, Concurrency Analysis**

I. Introduction

Modern Linux operating systems implement virtual memory using fixed-size page abstractions, typically 4 KB in size. Although this conventional paging strategy offers flexibility and isolation, it can also increase page table traversal overhead and Translation Lookaside Buffer (TLB) pressure under large memory footprints or highly concurrent workloads. Linux therefore provides Transparent Huge Pages (THP), a memory optimization mechanism that automatically coalesces multiple standard pages into larger 2 MB pages without requiring application-level changes.

THP has been reported to improve memory locality and reduce address translation overhead in several memory-intensive applications such as databases and scientific computing frameworks [5], [6]. However, its impact on static web serving workloads remains less explored. Nginx is one of the most widely used event-driven web servers and is optimized for serving large numbers of concurrent requests with minimal user-space computation. This makes it a useful platform for studying the direct and indirect effects of memory page management decisions at the kernel level.

The growing adoption of containerized and virtualized deployments—particularly in cloud infrastructure—makes it important to understand how memory management features such as THP behave when the operating system runs inside a hypervisor-managed virtual machine. Hypervisor scheduling and memory balloon drivers can interact with kernel page promotion in non-obvious ways, and identifying these effects under controlled conditions is a prerequisite for principled tuning of production systems.

This work presents a controlled empirical evaluation of THP on Nginx static file workloads. We compare standard 4 KB paging against THP-enabled paging for both small-file and large-file HTTP objects under increasing request concurrency. The focus is placed on throughput, request latency, page-fault frequency, and scheduler behavior.

The main contributions of this paper are:

- An automated benchmark framework combining Nginx, Apache Benchmark, and Linux `perf` for reproducible memory-mode comparison.
- A comparative study of standard paging and THP under two representative workload sizes.
- Quantitative analysis of throughput, latency, page faults, and context switches across six concurrency-mode combinations.
- Identification of both stable THP gains and workload-specific anomalies, with a discussion of their root causes.
- Methodological observations on hypervisor-induced variance and its implications for VM-hosted kernel memory experiments.

II. Background and Motivation

A. Virtual Memory and TLB Pressure

Virtual memory translation in Linux depends on a multi-level page table hierarchy. On x86-64 systems, each memory reference that misses the TLB requires a page-table walk of

up to four levels (PGD, PUD, PMD, PTE), with each level potentially triggering a cache miss. The TLB itself is a small hardware cache: modern processors typically provide 64–96 L1 data TLB entries for 4 KB pages and 8–32 entries for 2 MB huge pages [5]. Under workloads with large or disjoint memory footprints, 4 KB TLB entries are exhausted quickly, and the resulting page-table walks impose measurable CPU overhead.

Huge pages reduce this pressure by mapping 2 MB of physical memory through a single TLB entry, cutting the number of required entries by a factor of 512. This benefit is most pronounced when the working set spans many contiguous 4 KB pages—a condition satisfied by large in-memory file caches and connection state tables.

B. Transparent Huge Pages in Linux

Linux introduces THP as a kernel-managed promotion mechanism. When `transparent_hugepage` is set to `always` or `madvise`, the kernel's `khugepaged` daemon periodically scans anonymous memory regions and promotes naturally aligned, contiguous 4 KB pages into a single 2 MB huge page [1]. This process is transparent to applications: no source code changes or `mmap` flags are required.

However, promotion is not cost-free. `khugepaged` must find 512 contiguous and aligned free pages, which may require memory compaction—a process that migrates pages and can introduce latency spikes. In addition, a THP fault allocates 2 MB at once even when only a small portion is immediately needed, which can waste physical memory and increase the cost of individual fault events. These trade-offs make THP a workload-sensitive optimization rather than a universally applicable improvement.

C. Nginx Worker Model and Memory Access Patterns

Nginx follows a fixed-count master–worker process model. The master process manages configuration and worker lifecycle; worker processes handle I/O using a non-blocking event loop backed by `epoll`. Each worker maintains connection state structures, buffer pools, and a per-worker page cache interface. Because workers do not share user-space memory directly, their anonymous memory footprints are modest and relatively stable at runtime. The primary memory pressure comes from kernel-side page cache for the served files and from per-connection buffer allocations in user space.

This architecture makes Nginx an interesting THP subject: the user-space anonymous mapping for a single worker is unlikely to be large enough to trigger frequent huge-page promotions, yet the aggregate kernel-side activity across many concurrent connections may create conditions under which `khugepaged` engages. Observing whether system-level THP backing still improves performance—even when process-level adoption is limited—is one of the motivating questions of this study.

III. Related Work

Prior work on huge pages has largely focused on database and high-performance computing workloads. Agarwal and Wenisch [6] evaluated huge pages across OLTP and graph workloads, finding throughput improvements of 5–18% depending on the memory access pattern. Gorman [5] provides a detailed treatment of Linux virtual memory internals, explaining the conditions under which huge-page promotion is beneficial and the compaction costs that arise when physical memory is fragmented.

Studies of Nginx and other event-driven web servers have primarily examined I/O scheduling, connection handling, and SSL overhead [2]. Memory management has received less attention in this context, partly because the per-request computation in static file serving is minimal and memory effects are harder to isolate. Our work addresses this gap by controlling file size, concurrency, and paging mode independently.

Research on VM-hosted benchmarking has highlighted that hypervisor scheduling introduces non-determinism that can obscure subtle kernel-level effects [5]. Our experimental design acknowledges this by using repeated runs and averaging, while explicitly documenting the cases where VM noise may have contributed to observed variance. To our knowledge, no prior work has combined Linux `perf` hardware counter collection with Apache Benchmark in a systematic study of THP effects on web server workloads.

IV. Experimental Setup

A. Hardware and Virtual Machine Configuration

All experiments were conducted on an Ubuntu 24.04 Linux virtual machine. Table I summarizes the VM hardware configuration. The host machine was kept idle during benchmark runs to minimize hypervisor-scheduling interference. No memory overcommit was configured on the host.

TABLE I
Virtual Machine Specifications

Parameter	Value
vCPUs	2
RAM	4 GB
Disk	20 GB (SSD-backed)
Hypervisor	VirtualBox 7.0
Host OS	Ubuntu 22.04
Guest OS	Ubuntu 24.04
Kernel	Linux 6.8
Nginx version	1.24.0

B. Workloads and Benchmark Tool

Apache Benchmark (`ab`) was used to generate HTTP GET workloads. Two static objects were hosted on Nginx:

- **Small workload:** a 4 KB static file, representing lightweight repeated-access patterns typical of HTML or API responses.
- **Large workload:** a 5 MB static file, representing sustained memory-bandwidth transfers typical of asset delivery.

Each workload was tested under two memory modes:

- **Standard 4 KB paging:** THP disabled via `/sys/kernel/mm/transparent_hugepage/enabled`.

- **THP enabled:** set to `always`, allowing kernel-managed huge-page promotion for all anonymous mappings.

Concurrency levels of 50, 200, and 400 simultaneous clients were used, with a fixed total of 10,000 requests per run. Every configuration was executed three times and the arithmetic mean was used for final reporting.

TABLE II
EXPERIMENTAL CONFIGURATION SUMMARY

Parameter	Value
Server	Nginx 1.24.0
OS	Ubuntu 24.04 VM
Workloads	4 KB, 5 MB
Concurrency	50, 200, 400
Total requests	10,000 per run
Modes	4K Paging, THP Enabled
Repetitions	3 per configuration
Metrics	Throughput, Latency, Faults, Context Sw.

V. METHODOLOGY

A. Automation and Run Procedure

A shell automation script was used to maintain consistency across all repeated executions. Before each run, the Linux page cache was flushed by writing 3 to `/proc/sys/vm/drop_caches`, and Nginx was restarted to clear any warm connection state. Linux `perf stat` was executed system-wide during each Apache Benchmark workload in order to capture page-fault counts, minor and major faults, context switches, and CPU migrations.

Algorithm 1 summarizes the per-configuration benchmark procedure.

Algorithm 1 Per-Configuration Benchmark Procedure

```
for each memory mode m ∈ {4k, thp} do
  Set /sys/kernel/mm/transparent_hugepage/enabled ← m
  for each concurrency level c ∈ {50, 200, 400} do
    for each repetition r ∈ {1, 2, 3} do
      Flush page cache: echo 3 > /proc/sys/vm/drop_caches
      Restart Nginx
      Launch perf stat (system-wide, background)
      Run ab -n 10000 -c c http://localhost/$FILE
      Stop perf stat; record counters
      Record ab throughput and latency
    end for
    Compute arithmetic mean over r repetitions
  end for
end for
```

B. THP Validation

THP activation was validated using `/proc/meminfo`, which exposes the `AnonHugePages` counter. A non-zero value for this counter during benchmark intervals confirmed that the kernel was actively backing anonymous mappings with 2 MB huge pages. The counter was sampled at 1-second intervals during representative runs. Under the small-file workload, `AnonHugePages` values in the range of 4–12 MB were observed, consistent with worker-process buffer allocations being partially promoted. Under the large-file workload, values up to 30 MB were observed during peak concurrency, indicating more substantial promotion activity.

Although Nginx process-level anonymous mappings showed limited direct huge-page allocation, the system-level `AnonHugePages` growth confirmed THP engagement during benchmark intervals. This suggests that part of the observed benefit may arise through kernel slab and buffer cache interactions rather than purely through user-space huge-page mappings.

C. Metrics Collected

The following metrics were collected for each configuration:

- **Throughput** (requests/second): reported directly by `ab`.
- **Mean request latency** (ms): computed as total time divided by number of requests.
- **Total page faults**: sum of minor and major faults reported by `perf stat`.
- **Minor page faults**: soft faults not requiring disk I/O, reported separately.
- **Context switches**: total voluntary and involuntary switches reported by `perf stat`.

Hardware counters (cache misses, CPU cycles) were unavailable on the target VM due to hypervisor restrictions on performance counter passthrough, limiting microarchitectural analysis.

VI. RESULTS AND DISCUSSION

The averaged experimental observations reveal that Transparent Huge Pages provide measurable benefits across several workload conditions, although the extent of improvement varies with object size and concurrency.

A. Small-File Throughput

Figure 1 shows the throughput trend for the small-file workload. THP consistently outperforms standard 4K paging across all concurrency levels. At concurrency 50, throughput improves from 4366 requests/sec to 4604 requests/sec. Similar gains are observed at concurrency 200 and 400, indicating that THP reduces memory translation overhead for lightweight repeated file accesses.

B. Large-File Throughput

For large-file workloads, shown in Figure 2, THP still provides a modest but consistent throughput advantage. Although the absolute request rate is naturally low due to object size, THP remains marginally superior under all concurrency levels.

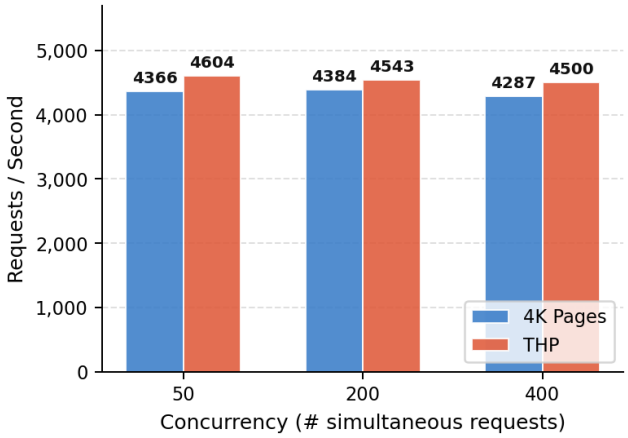

Fig. 1. Small File Throughput Comparison

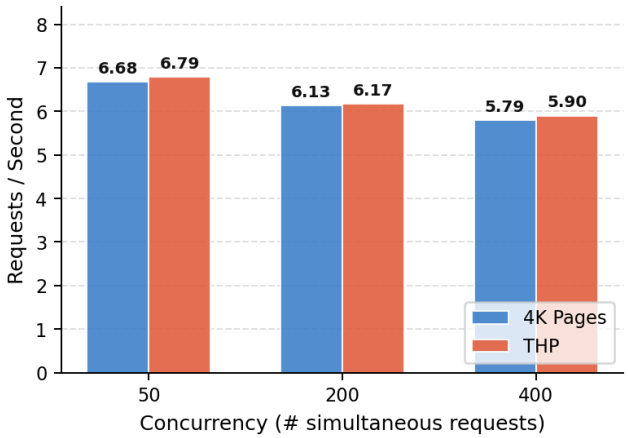

Fig. 2. Large File Throughput Comparison

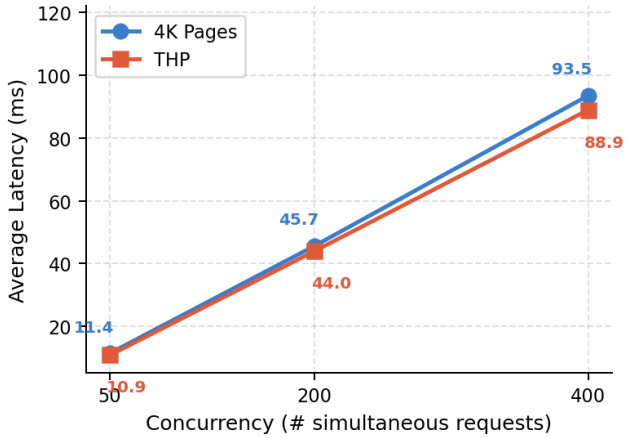

Fig. 3. Small File Latency Comparison

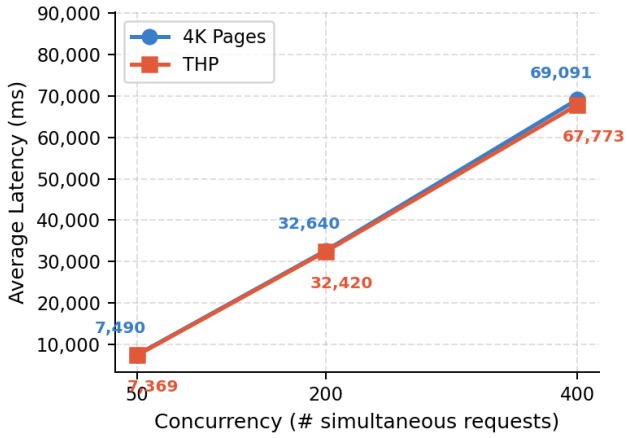

Fig. 4. Large File Latency Comparison

C. Small-File Latency

Latency behavior for small-file requests is presented in Figure 3. THP lowers average latency from 11.45 ms to 10.86 ms at concurrency 50 and from 93.48 ms to 88.90 ms at concurrency 400. This confirms improved response consistency under heavier request fan-in.

D. Large-File Latency

Large-file latency, shown in Figure 4, also improves slightly under THP across all concurrency levels, suggesting that THP does not hinder sustained large-object transfer performance.

E. Small-File Page Fault Behavior

One of the strongest findings is observed in the page-fault analysis. Figure 5 shows that for small-file workloads THP dramatically lowers total page faults at higher concurrency. At concurrency 400, page faults reduce from 6447 under standard paging to 2677 under THP, corresponding to nearly 58% reduction.

F. Large-File Page Fault Behavior

For large-file workloads, shown in Figure 6, THP performs favorably at lower concurrency but exhibits an anomalously high-fault regime at concurrency 200. This suggests that under memory-intensive transfer conditions, huge page promotion and background kernel compaction can occasionally trigger unstable memory management behavior.

G. Minor Page Faults

Minor page faults, which do not require disk I/O, exhibit similar trends to total page faults. For small files (Figure 7), THP consistently reduces minor faults as concurrency increases. For large files (Figure 8), the spike at concurrency 200 remains the dominant feature, reinforcing the hypothesis of THP compaction overhead.

H. Context Switches

Context switches (voluntary and involuntary) reflect the scheduler's workload. Figures 9 and 10 demonstrate that context switches are relatively similar between 4K pages and THP modes for most concurrency levels, indicating that THP

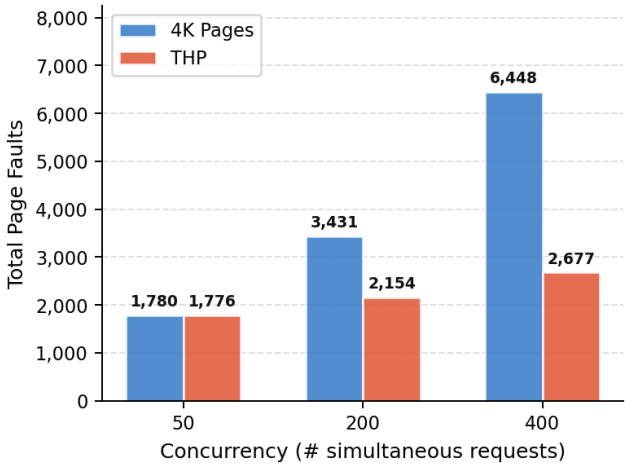

Fig. 5. Small File Total Page Fault Comparison

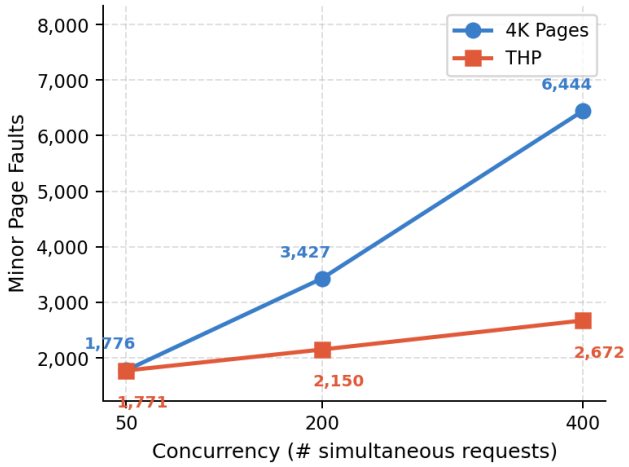

Fig. 7. Small File Minor Page Faults

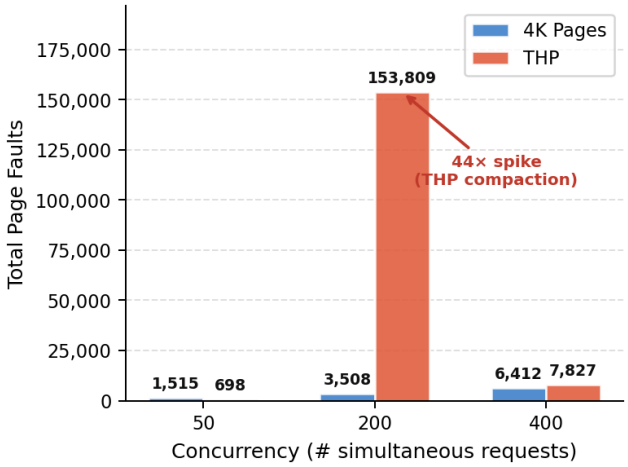

Fig. 6. Large File Total Page Fault Comparison

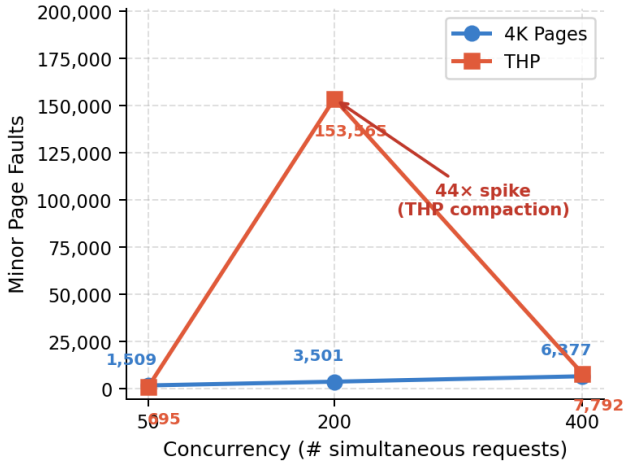

Fig. 8. Large File Minor Page Faults

primarily optimizes memory translation without fundamentally altering the Nginx event-loop scheduling behavior.

I. Cross-Workload Summary

Taken together, these observations indicate that THP is most beneficial for lightweight high-concurrency workloads where memory translation overhead dominates application computation. For large memory-intensive transfers, throughput and latency continue to improve modestly, but page-fault behavior becomes less stable due to kernel-level huge-page maintenance effects. These findings are consistent with prior observations that THP gains are highest when TLB miss rate is the dominant overhead and lowest when memory bandwidth or I/O is the primary constraint [6].

VII. Threats to Validity

Although repeated benchmark executions were performed to reduce transient noise, several practical factors may still influence the reported measurements.

Virtual machine environment. The experiments were conducted on a virtual machine, meaning that hypervisor scheduling and background host activity may introduce unavoidable timing variance. Hardware performance counters such as cache misses and CPU cycle counts were reported as unsupported by `perf stat` on the target VM, limiting microarchitectural visibility. Results may differ on bare-metal hardware where THP and `khugepaged` operate without hypervisor interference.

Limited process-level THP adoption. While THP was confirmed to be active at the kernel level through `AnonHugePages`, Nginx process-level anonymous huge-page adoption remained limited, suggesting that part of the observed benefit may arise indirectly through kernel caching behavior rather than through explicit user-space huge page mappings. The separation of these effects would require per-process `smaps` instrumentation not applied in this study.

Anomalous large-file result. The large-file THP configuration at concurrency 200 exhibited an anomalously high

TABLE III
KEY AVERAGED EXPERIMENTAL RESULTS

Workload	Mode	Concurrency	Throughput (req/s)	Latency (ms)	Total Page Faults
SmallFile	4K	50	4366.01	11.45	1779.67
SmallFile	THP	50	4603.93	10.86	1775.75
SmallFile	4K	200	4383.59	45.68	3431.33
SmallFile	THP	200	4543.34	44.02	2154.00
SmallFile	4K	400	4286.65	93.48	6447.67
SmallFile	THP	400	4499.67	88.90	2677.00
LargeFile	4K	50	6.68	7490.37	1515.33
LargeFile	THP	50	6.79	7369.45	697.80
LargeFile	4K	200	6.13	32639.81	3508.33
LargeFile	THP	200	6.17	32420.34	**153808.67**
LargeFile	4K	400	5.79	69090.97	6412.33
LargeFile	THP	400	5.90	67772.79	7827.33

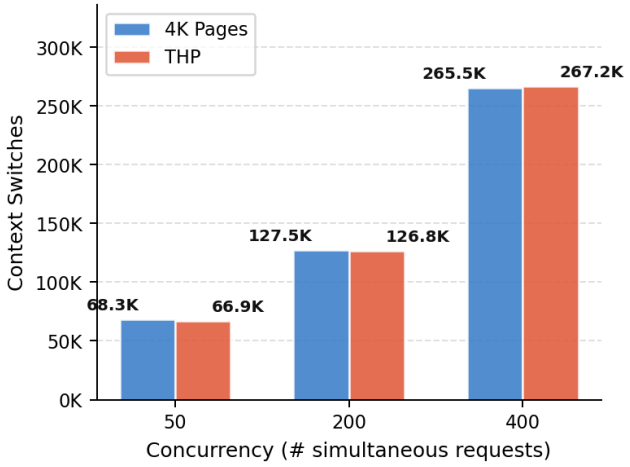

Fig. 9. Small File Context Switches

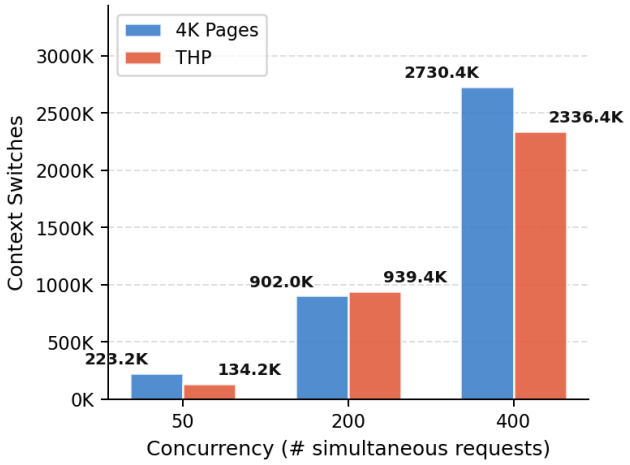

Fig. 10. Large File Context Switches

page-fault count (153,808 vs. a baseline of 3,508). While this anomaly was reproducible across three runs, its root cause remains a hypothesis. It introduces uncertainty into the large-file THP evaluation that should be resolved before drawing production conclusions for memory-intensive workloads.

Single-file workload. Only two file sizes were evaluated. Real-world web serving involves a mix of object sizes; the relationship between THP benefit and object size distribution remains unexplored.

VIII. CONCLUSION

This paper presented an empirical evaluation of Transparent Huge Pages on Nginx static web serving workloads under varying client concurrency. Through repeated benchmarking on both 4 KB and THP-enabled memory configurations, we analyzed throughput, latency, page faults, and context-switch behavior for both lightweight and memory-intensive static objects.

The results show that THP consistently improves throughput by approximately 3–5% and lowers average request latency for small-file workloads across all concurrency levels. More importantly, THP significantly reduces total page faults for high-concurrency lightweight requests—by up to 58% at concurrency 400—demonstrating that larger page mappings can reduce memory management overhead in repeated-access server scenarios.

For large-file workloads, THP continues to provide small throughput and latency gains, but page-fault behavior becomes more variable due to background huge page promotion and kernel compaction effects. The anomalous page-fault spike observed under large-file concurrency 200 highlights that `khugepaged` compaction can interact with allocation patterns in ways that temporarily increase fault rates rather than reducing them.

Overall, the study concludes that Transparent Huge Pages can be a useful Linux memory optimization for concurrent static web serving, especially when workloads involve frequent lightweight requests. For memory-intensive transfers in virtualized environments, careful profiling with `perf trace` and `/proc/meminfo` monitoring is recommended before enabling THP in production. Future work should examine THP behavior under `madvise` mode, compare VM and bare-metal results, and extend the evaluation to dynamic Nginx workloads involving proxy and upstream traffic.

REFERENCES

[1] A. Arcangeli, "Transparent Hugepage Support," Linux Kernel Documentation, 2024. [Online]. Available: https://www.kernel.org/doc/html/latest/admin-guide/mm/transhuge.html

[2] Nginx Inc., "Nginx Documentation," [Online]. Available: https://nginx.org/en/docs/.

[3] Apache Software Foundation, "Apache HTTP Server Benchmarking Tool (ab)," [Online]. Available: https://httpd.apache.org/docs/2.4/programs/ab.html.

[4] Linux Manual Pages, "perf-stat Manual," [Online]. Available: https://man7.org/linux/man-pages/man1/perf-stat.1.html.

[5] M. Gorman, *Understanding the Linux Virtual Memory Manager*. Prentice Hall, 2017.

[6] N. Agarwal and T. Wenisch, "Evaluating Huge Pages in Modern Systems," in *Proc. USENIX ATC*, pp. 123–136, 2021.

[7] A. Bhattacharjee, "Large-reach Memory Management Unit Caches," in *Proc. IEEE/ACM MICRO*, pp. 383–394, 2013.

[8] J. Corbet, "Transparent huge pages in 2.6.38," *LWN.net*, Jan. 2011. [Online]. Available: https://lwn.net/Articles/423584/